

Submission Worksheet

Submission Data

Course: IT202-010-S2026

Assignment: IT202 Milestone 1 - 2026

Student: Ryan C. (rc728)

Status: Graded | **Worksheet Progress:** 100%

Potential Grade: 10.00/10.00 (100.00%)

Received Grade: 10.00/10.00 (100.00%)

Started: 4/12/2026 5:14:36 PM

Updated: 4/13/2026 11:43:31 PM

Grading Link: <https://learn.ethereallab.app/assignment/v3/IT202-010-S2026/it202-milestone-1-2026/grading/rc728>

View Link: <https://learn.ethereallab.app/assignment/v3/IT202-010-S2026/it202-milestone-1-2026/view/rc728>

Instructions

- Overview Link: <https://youtu.be/IDtqdaLwcVg>
- 1. Refer to Milestone1 of this doc: <https://docs.google.com/document/d/1XE96a8DQ52Vp49XACBDTNCq0xYDt3kF29cO88EWVwfo/view>
- 2. Ensure you read all instructions and objectives before starting.
- 3. Ensure you've gone through each lesson related to this Milestone
- 4. Switch to the Milestone1 branch
 - 1. `git checkout Milestone1` (ensure proper starting branch)
 - 2. `git pull origin Milestone1` (ensure history is up to date)
- 5. Fill out the below worksheet
 - Ensure there's a comment with your UCID, date, and brief summary of the snippet in each screenshot
 - Ensure proper styling is applied to each page
 - Ensure there are no visible technical errors; only user-friendly messages are allowed
- 6. Once finished, click "Submit and Export"
- 7. Locally add the generated PDF to a folder of your choosing inside your repository folder and move it to Github
 - 1. `git add .`
 - 2. `git commit -m "adding PDF"`
 - 3. `git push origin Milestone1`
 - 4. On Github merge the pull request from Milestone1 to qa
 - 5. On Github create a pull request from qa to prod and immediately merge. (This will trigger the prod deploy to make the heroku prod links work)
- 8. Upload the same PDF to Canvas
- 9. Sync Local
- 10. `git checkout qa`
- 11. `git pull origin qa`

Section #1: (2 pts.) Feature: User Will Be Able To Register A New Account

Task #1 (0.67 pts.) - Validation

Details:

- Show the relevant code snippets for each validation layer
- Show the relevant demo of each validation layer
- Briefly explain how the validation steps work at each layer

Task #1 (0.33 pts.) - HTML Form/Validation

Details:

- System should allow the user to correct the error without wiping/clearing the form (for the PHP side this includes sticky-form logic to keep the username/email address entries)

Part 1:

Details:

- Show the code related to the register page's form (HTML validation)
- Show examples of each validation message (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)

The screenshot shows a web form titled "Register" on a yellow background. It has two input fields: "Email" containing "abc" and "Confirm" containing "*****". A red error message box is displayed above the "Confirm" field, stating: "Please include an '@' in the email address. 'abc' is missing an '@'." Below the inputs is a "Register" button.

this shows an email error

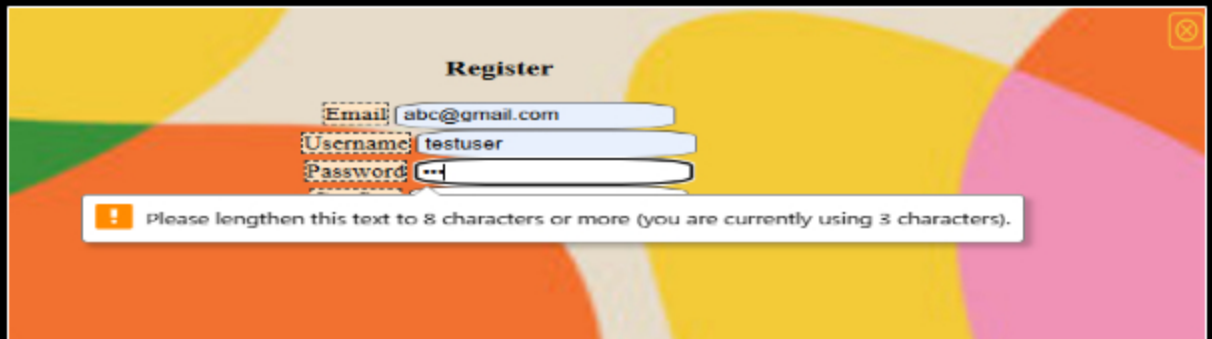
```

<div>Registers/his
<form onSubmit="return validate(this)" method="POST">
  <div>
    <label for="email">Email</label>
    <input id="email" type="email" name="email" required value "<php echo htmlspecialchars($email ?? ''); >" />
  </div>
  <div>
    <label for="username">Username</label>
    <input type="text" name="username" required maxlength="20" value "<php echo htmlspecialchars($username ?? ''); >" />
  </div>
  <div>
    <label for="pw">Password</label>
    <input type="password" id="pw" name="password" required minlength="8" />
  </div>
  <div>
    <label for="confirm">Confirm</label>
    <input type="password" name="confirm" required minlength="8" />
  </div>
  <button type="submit">Register</button>
</form>

```

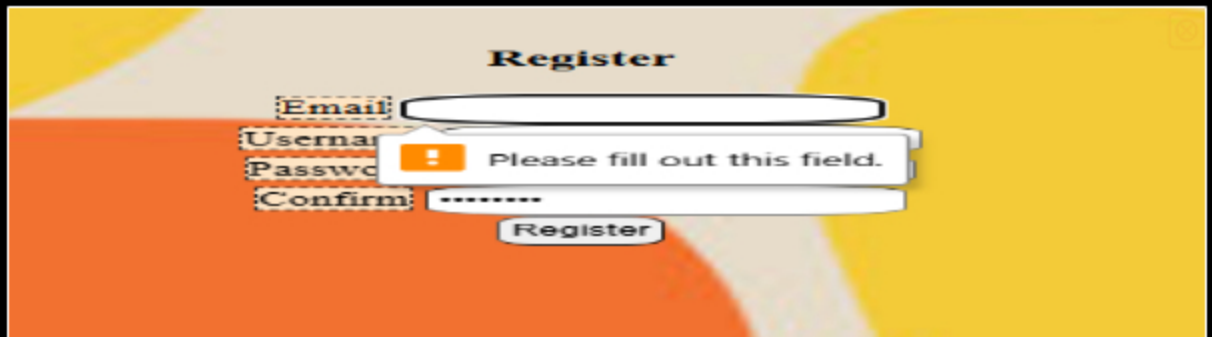
```
</div>  
<input type="submit" value="Register" />  
</form>
```

this shows the code which allows for the username and email to stay after an error



The screenshot shows a 'Register' form with three input fields: 'Email' containing 'abc@gmail.com', 'Username' containing 'testuser', and 'Password' which is empty. A red error message box is displayed below the password field, stating: 'Please lengthen this text to 8 characters or more (you are currently using 3 characters)'. The background is a colorful abstract pattern.

invalid password error



The screenshot shows a 'Register' form with four input fields: 'Email', 'Username', 'Password', and 'Confirm'. The 'Email' field is empty, and a red error message box is displayed over it, stating: 'Please fill out this field.'. The 'Username' field contains 'testuser', 'Password' contains '12345678', and 'Confirm' contains '12345678'. A 'Register' button is at the bottom. The background is a colorful abstract pattern.

empty field error

 Saved: 4/13/2026 11:20:25 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

The register form uses both HTML and PHP validation to be able to prevent errors. The HTML validation is used to be able to check the input before it is submitted and give feedback. The PHP validation is used on the server side to be able to check for email format, username rules, password length, matching passwords, or duplicate accounts. To keep the form information still inside without wiping it after an error uses sticky form logic which was implemented into the email and the username field. It uses `value="" />` for the email and `value="" />` for the username. This allows for the user's input to stay in the form after the error allowing the user to correct mistakes without needing to type back all the information.

 Saved: 4/13/2026 11:20:25 PM

☰ Task #2 (0.33 pts.) - JS Validation (validate() function)

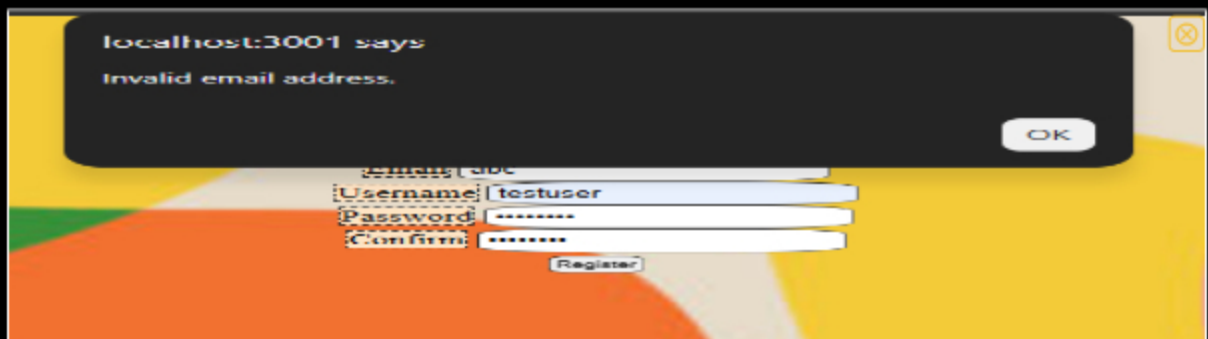
Progress: 100%

📁 Part 1:

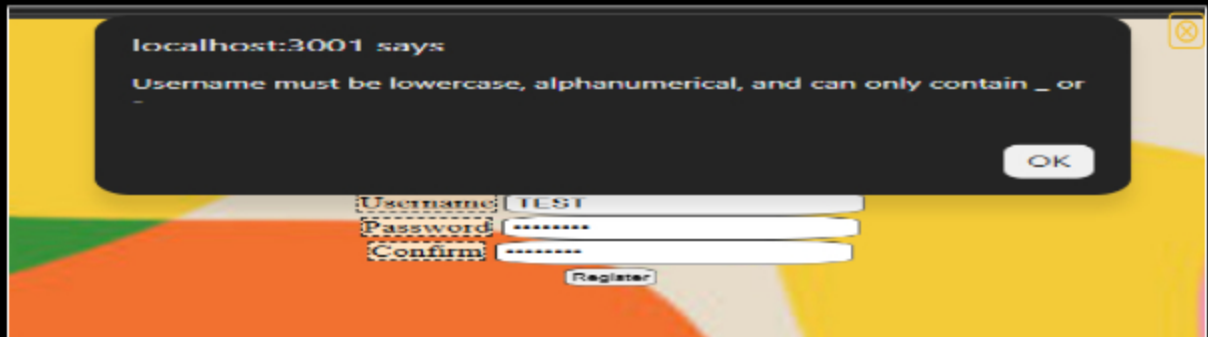
Progress: 100%

Details:

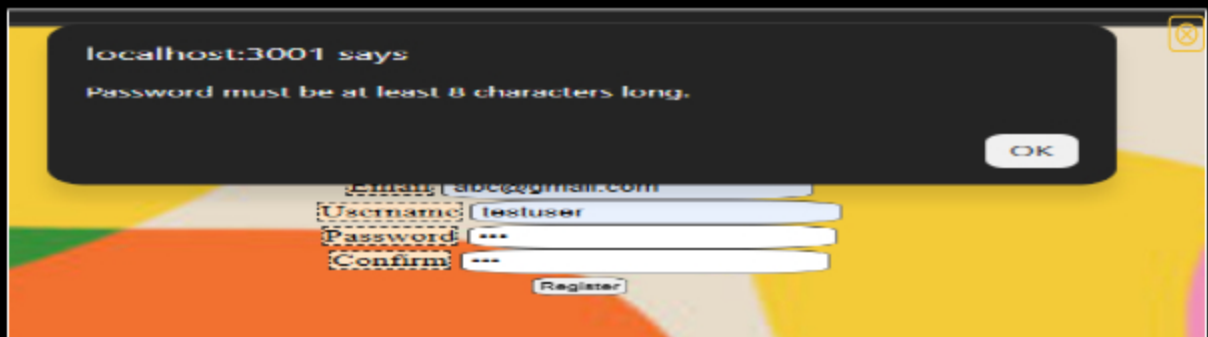
- Show the code related to the register page's validate() function
- Show examples of each validation message [username format, email format, password format, password doesn't match confirm](you may be able to capture this in one or few screenshots)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag



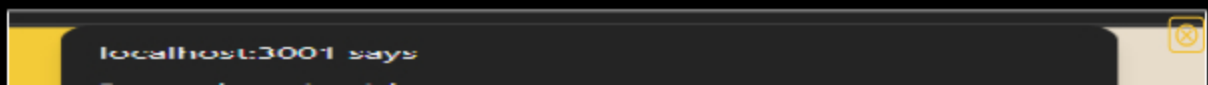
invalid email error



user must be lowercase error



password length error



passwords must match.

OK

Email: abc@gmail.com

Username: testuser

Password: *****

Confirm: *****

Register

password must match error

```

<script>
    function validate(frm) {
        // 1. Email format
        if (!validateEmail(frm.email.value)) {
            alert("Invalid email format. Please use a valid email address.");
            return false;
        }

        // 2. Username format
        if (!validateUsername(frm.username.value)) {
            alert("Invalid username format. Username must be at least 3 characters long.");
            return false;
        }

        // 3. Password length
        if (frm.password.value.length < 8) {
            alert("Password must be at least 8 characters long.");
            return false;
        }

        // 4. Passwords must match
        if (frm.password.value != frm.confirm.value) {
            alert("Passwords do not match. Please re-enter your password correctly.");
            return false;
        }

        // 5. All checks passed
        return true;
    }

    function register() {
        if (validate(frm)) {
            // If validation passes, proceed with registration
            // (This part would typically send data to a server)
        } else {
            // If validation fails, alert user and stop submission
            return false;
        }
    }

    // Attach event listener to the Register button
    document.getElementById("register").addEventListener("click", register);

```

this shows the js validation code

 Saved: 4/13/2026 11:11:17 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

The validate() function uses JS validation before the form is submitted where it checks for valid email format, username format, password length, and if the passwords are matching. If any of these checks fails, there will be an alert pop up shown and the form submission is stopped. the allows for errors to be caught before data is sent to the server.

 Saved: 4/13/2026 11:11:17 PM

Task #3 (0.33 pts.) - PHP Validation (steps before the DB call)

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the code related to the register page's PHP validation
- Show examples of each validation message [username format, email format, password format, invalid password, password doesn't match confirm, username taken, email taken] (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag and temporarily override the `validate()` function or use the provided helper script in the instructions
- Include the outputs related to username/email already in use

```

<?php
// ... (previous code) ...

// Validation functions
function validate_email($email) {
    return filter_var($email, FILTER_VALIDATE_EMAIL);
}

function validate_username($username) {
    return preg_match('/^[a-z0-9_]{3,20}$/', $username);
}

function validate_password($password) {
    return preg_match('/^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*^&])[8-100]$/s', $password);
}

// Register function
function register($username, $email, $password, $confirm_password) {
    // Check if email is already taken
    if (check_email($email)) {
        return "Email already taken";
    }

    // Check if username is already taken
    if (check_username($username)) {
        return "Username already taken";
    }

    // Validate email
    if (!validate_email($email)) {
        return "Invalid email address";
    }

    // Validate username
    if (!validate_username($username)) {
        return "Username must be lowercase, alphanumeric, and can only contain _ or -";
    }

    // Validate password
    if (!validate_password($password)) {
        return "Password must be at least 8 characters long";
    }

    // Check if passwords match
    if ($password !== $confirm_password) {
        return "Passwords do not match";
    }

    // If all validations pass, register the user
    // ... (registration logic) ...
}

```

this shows the code for the php validation

Invalid email address

Register

Email:

Username:

Password:

Confirm:

invalid email address error

Username must be lowercase, alphanumeric, and can only contain _ or -

Register

Email:

Username:

Password:

Confirm:

username must be lowercase error

Password must be at least 8 characters long

Register

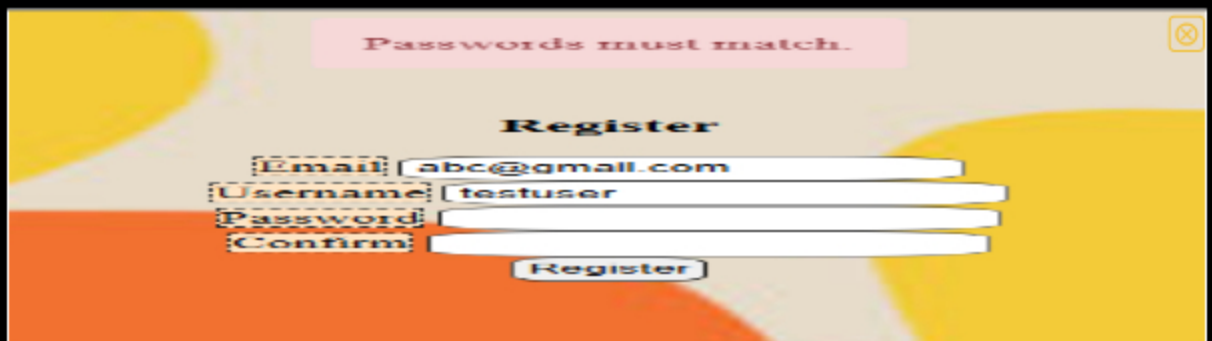
Email:

Username:

Password:

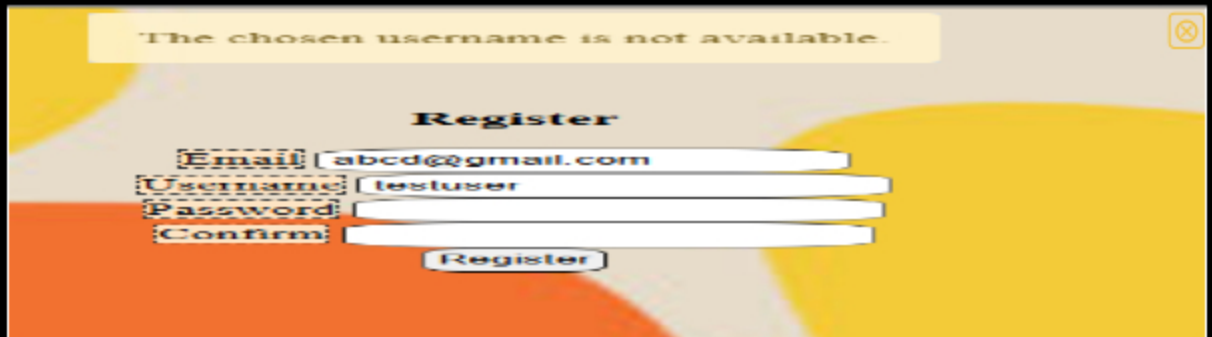
Confirm:

password must be at least 8 characters error



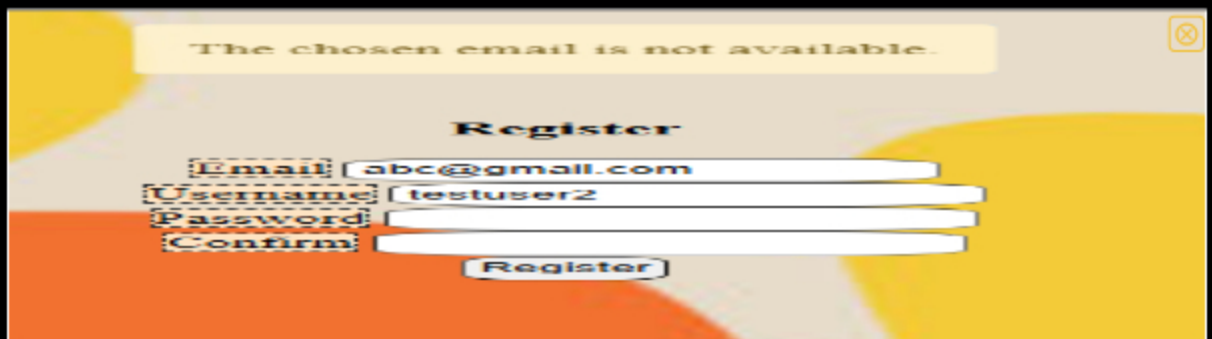
A registration form titled "Register" with fields for Email, Username, Password, and Confirm. The Email field contains "abc@gmail.com", Username contains "testuser", and both Password and Confirm fields contain "1234567". A red error message "Passwords must match." is displayed at the top right. A "Register" button is at the bottom.

password must match error



A registration form titled "Register" with fields for Email, Username, Password, and Confirm. The Email field contains "abcd@gmail.com", Username contains "testuser", and both Password and Confirm fields contain "1234567". A yellow error message "The chosen username is not available." is displayed at the top left. A "Register" button is at the bottom.

username not available error



A registration form titled "Register" with fields for Email, Username, Password, and Confirm. The Email field contains "abc@gmail.com", Username contains "testuser2", and both Password and Confirm fields contain "1234567". A yellow error message "The chosen email is not available." is displayed at the top left. A "Register" button is at the bottom.

email not available error



Saved: 4/13/2026 11:11:14 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

The register page uses PHP validation before submission to check for valid email format, username format, password length, empty fields, and matching passwords. It also checks for

duplicate usernames and emails using database constraints. If any validation fails, a flash message is displayed and the user is not inserted into the database. This allows for invalid or duplicate data to be caught on the server before being stored.



Saved: 4/13/2026 11:11:14 PM

Task #2 (0 / 0.67 pts.) - Related URLs

Progress: 100%

Details:

- Include the direct link to this file from the Milestone branch (should end in `.php` , , zero credit if it does not)
- Include the render.com prod link to this file (Just grab the base prod url and manually enter the path to the file)
- Include the related pull request link for this feature

URL #1

https://github.com/524RC/RC728-IT202-010/blob/prod/public_html/project/register.php



URL

https://github.com/524RC/RC728-IT202-010/blob/prod/public_html/project/register.php



URL #2

<https://rc728-it202-010-prod.onrender.com/project/register.php>



URL

<https://rc728-it202-010-prod.onrender.com/project/register.php>



URL #3

<https://github.com/524RC/RC728-IT202-010/pull/23>



URL

<https://github.com/524RC/RC728-IT202-010/pull/23>



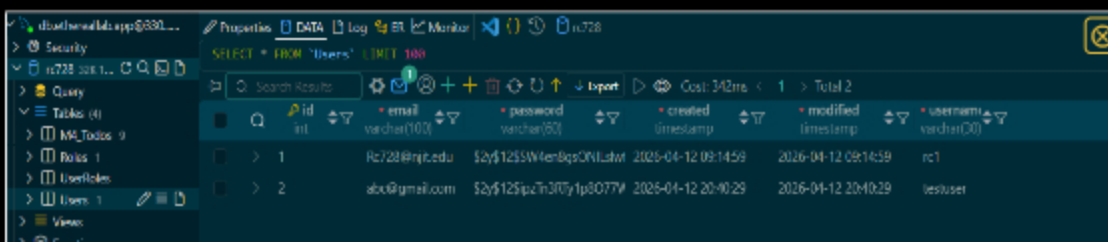
Saved: 4/13/2026 11:41:53 PM

Task #3 (0 / 0.67 pts.) - Database - Users table

Progress: 100%

Details:

- Add a screenshot of the database view from vs code showing a valid registered user (preferably a recent one during evidence capturing)



id	email	password	created	modified	username
1	RC728@rc.edu	\$2y\$12\$3W4en8q3QNL4m	2025-04-12 09:14:59	2026-04-12 09:14:59	rc1
2	abc@gmail.com	\$2y\$12\$5p4n307y1p8077M	2025-04-12 20:40:29	2026-04-12 20:40:29	testuser

this shows the database



Saved: 4/13/2026 11:43:31 PM

Section #2: (2 pts.) Feature: User Will Be Able To Login To Their Account

Progress: 100%

≡ Task #1 (0.67 pts.) - Validation

Progress: 100%

Details:

- Show the relevant code snippets for each validation layer (html, js, php)
- Show the relevant demo of each validation layer (html, js, php)
- Briefly explain how the validation steps work at each layer

≡ Task #1 (0.33 pts.) - HTML Form/Validation

Progress: 100%

Details:

- System should allow the user to correct the error without wiping/clearing the form (for the PHP side this includes sticky-form logic to keep the username/email address entries)

📄 Part 1:

Progress: 100%

Details:

- Show the code related to the login page's form (HTML validation)
- Show examples of each validation message (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)

```
<h3>Login</h3>
<form onsubmit="return validate(this)" method="POST" >
  <div>
    <label for="email">Email or Username</label>
    <input id="email" type="text" name="email" required value="<?php echo htmlspecialchars($_POST['email']) ?? ''; ?>" />
  </div>
  <div>
    <label for="pw">Password</label>
    <input type="password" id="pw" name="password" required minlength="8" />
  </div>
  <input type="submit" value="Login" />
</form>
```

```
</div>  
<input type="password" id="pw" name="password" required minlength="8" />  
<input type="submit" value="Login" />  
</form>
```

this shows the html validation code

A login form titled "Login" with a yellow and orange background. It has two input fields: "Email or Username" and "Password". The "Email or Username" field is empty, and a red dashed border highlights it. A red error message box with an exclamation mark icon says "Please fill out this field." The "Password" field has a red dashed border and contains three dots, indicating it is required but not yet filled.

This shows an missing username or email error

A login form titled "Login" with a yellow and orange background. The "Email or Username" field contains "abc@gmail.com" and has a red dashed border. The "Password" field is empty and has a red dashed border. A red error message box with an exclamation mark icon says "Please fill out this field."

this shows a missing password error

A login form titled "Login" with a yellow and orange background. The "Email or Username" field contains "abc@gmail.com" and has a red dashed border. The "Password" field contains "abc" and has a red dashed border. A red error message box with an exclamation mark icon says "Please lengthen this text to 8 characters or more (you are currently using 3 characters)."

this shows a too short password error



Saved: 4/13/2026 3:13:17 AM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

The HTML validation works by using the required attribute on both the username/email and password field to make it so the user cannot submit the form if either field is empty. The password field also includes that minlength="8" to make it so the minimum password length is 8. These validations are handled by the browser and display an error message. This allows the user to fix mistakes before submission.



Saved: 4/13/2026 3:13:17 AM

Task #2 (0.33 pts.) - JS Validation (validate() function)

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the code related to the login page's validate() function
- Show examples of each validation message [username format, email format, password format] (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag

```
function validate(form) {  
  // [555] 2. implement: function that validates form (i.e. do this on your own, outside the end of this form)  
  // [555] 3. return: false for an error and true for success  
  let login = form.querySelector('#login');  
  let password = form.querySelector('#password');  
  
  // [555] 4. validate: username  
  if (login.value.length < 3) {  
    alert('Username must be at least 3 characters long');  
    return false;  
  }  
  
  // [555] 5. validate: email  
  if (login.value.indexOf('@') < 0) {  
    alert('Email must contain an @ symbol');  
    return false;  
  }  
  
  // [555] 6. validate: password  
  if (password.value.length < 8) {  
    alert('Password must be at least 8 characters long');  
    return false;  
  }  
  
  // [555] 7. success  
  return true;  
}
```

this shows the js validation

localhost:3001 says

Password must be at least 8 characters long.

OK

Email or Username: abc@gmail.com

Password: ***

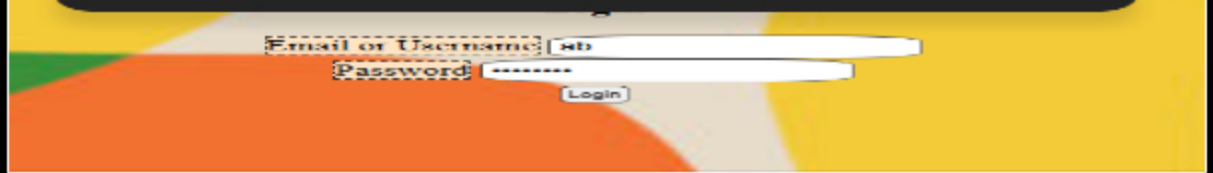
Login

This shows an invalid password error

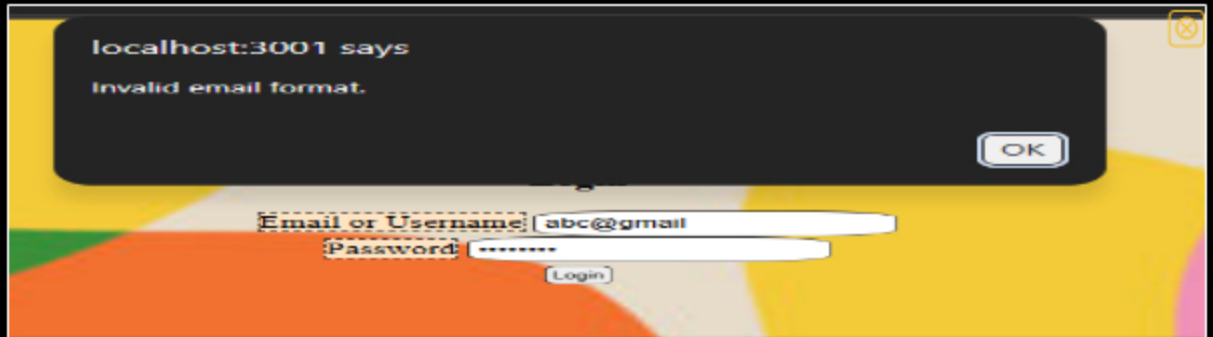
localhost:3001 says

Username must be at least 3 characters.

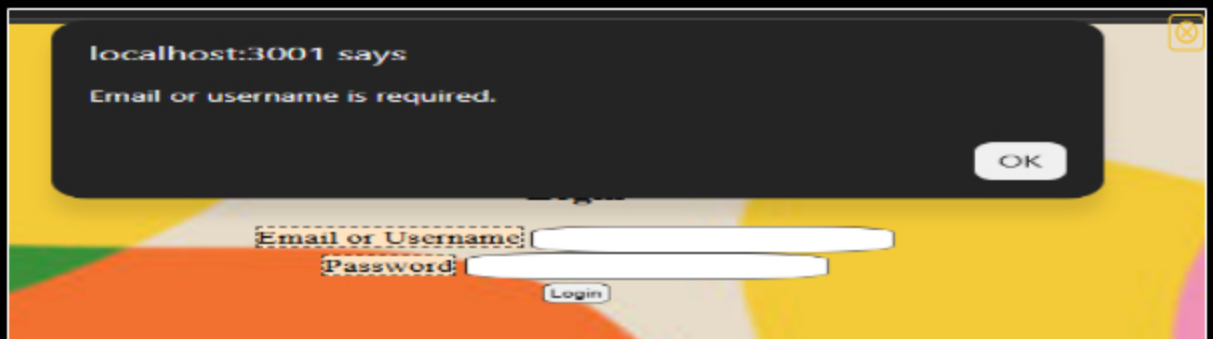
OK



this shows an Invalid username error



this shows an Invalid email error



this shows an empty password and username/email error

 Saved: 4/13/2026 8:53:44 PM

⇒ Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

The JS validation is handled by the `validate()` function which would run when a form is submitted. It checks if the email/username field is empty, if the input is a valid email format if it has a `@` and a valid username length, and if the password is atleast 8 characters long. if any of the checks fail, then an alert message is shown and the form submission is cancelled. This solves the requirement by validating user input on the client side and allowing users to fix errors before the form is submitted.

 Saved: 4/13/2026 8:53:44 PM

≡ Task #3 (0.33 pts.) - PHP Validation (steps before the DB call)

Progress: 100%

📁 Part 1:

Progress: 100%

Details:

- Show the code related to the login page's PHP validation
- Show examples of each validation message [username format, email format, password format, invalid password] (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag and temporarily override the `validate()` function or use the provided helper script in the instructions
- Include the outputs related to user doesn't exist and php-side password mismatch (the one that checks against the DB hash)

```
1 <?php
2 // ...
3 // ...
4 // ...
5 // ...
6 // ...
7 // ...
8 // ...
9 // ...
10 // ...
11 // ...
12 // ...
13 // ...
14 // ...
15 // ...
16 // ...
17 // ...
18 // ...
19 // ...
20 // ...
21 // ...
22 // ...
23 // ...
24 // ...
25 // ...
26 // ...
27 // ...
28 // ...
29 // ...
30 // ...
31 // ...
32 // ...
33 // ...
34 // ...
35 // ...
36 // ...
37 // ...
38 // ...
39 // ...
40 // ...
41 // ...
42 // ...
43 // ...
44 // ...
45 // ...
46 // ...
47 // ...
48 // ...
49 // ...
50 // ...
51 // ...
52 // ...
53 // ...
54 // ...
55 // ...
56 // ...
57 // ...
58 // ...
59 // ...
60 // ...
61 // ...
62 // ...
63 // ...
64 // ...
65 // ...
66 // ...
67 // ...
68 // ...
69 // ...
70 // ...
71 // ...
72 // ...
73 // ...
74 // ...
75 // ...
76 // ...
77 // ...
78 // ...
79 // ...
80 // ...
81 // ...
82 // ...
83 // ...
84 // ...
85 // ...
86 // ...
87 // ...
88 // ...
89 // ...
90 // ...
91 // ...
92 // ...
93 // ...
94 // ...
95 // ...
96 // ...
97 // ...
98 // ...
99 // ...
100 // ...
```

this shows the code for the php validation

```
1 // ...
2 // ...
3 // ...
4 // ...
5 // ...
6 // ...
7 // ...
8 // ...
9 // ...
10 // ...
11 // ...
12 // ...
13 // ...
14 // ...
15 // ...
16 // ...
17 // ...
18 // ...
19 // ...
20 // ...
21 // ...
22 // ...
23 // ...
24 // ...
25 // ...
26 // ...
27 // ...
28 // ...
29 // ...
30 // ...
31 // ...
32 // ...
33 // ...
34 // ...
35 // ...
36 // ...
37 // ...
38 // ...
39 // ...
40 // ...
41 // ...
42 // ...
43 // ...
44 // ...
45 // ...
46 // ...
47 // ...
48 // ...
49 // ...
50 // ...
51 // ...
52 // ...
53 // ...
54 // ...
55 // ...
56 // ...
57 // ...
58 // ...
59 // ...
60 // ...
61 // ...
62 // ...
63 // ...
64 // ...
65 // ...
66 // ...
67 // ...
68 // ...
69 // ...
70 // ...
71 // ...
72 // ...
73 // ...
74 // ...
75 // ...
76 // ...
77 // ...
78 // ...
79 // ...
80 // ...
81 // ...
82 // ...
83 // ...
84 // ...
85 // ...
86 // ...
87 // ...
88 // ...
89 // ...
90 // ...
91 // ...
92 // ...
93 // ...
94 // ...
95 // ...
96 // ...
97 // ...
98 // ...
99 // ...
100 // ...
```

this shows the code for the php validation after the DB call for incorrect passwords and non existent users

Username must be lowercase, alphanumerical, and can only contain _ or -

Login

Email or Username:

Password:

Login

invalid username format error

Invalid email address.

Login

Email or Username: abc@gmail

Password:

Login

invalid email format error

Password must be at least 8 characters long.

Login

Email or Username: abc@gmail.com

Password:

Login

invalid password error

Email/Username must not be empty.

Username must be lowercase, alphanumerical, and can only contain _ or -

Password must not be empty.

Login

Password must be at least 8 characters long.

Password:

Login

empty fields error

Invalid login attempt. Please check your email and password.

Login

Email or Username: abc@gmail.com

Password:

Login

invalid login attempt error



Saved: 4/13/2026 3:57:15 AM

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

the PHP validations runs after the form is submtited and before the login is completed. it checks whether the email/username field is empty, if the input should be treated as an email or a username, and if it is following the correct format. The password also validates if the input is atleast 8 characters long and if any of these validations fail, a flash message will be displayed and the login process is stopped. After validation, the system will check the database to see if the user exists and if the password matches using the function `password_verify()`. If either the user is non-existent or the password was incorrect, the both will return the same error message for security.



Saved: 4/13/2026 3:57:15 AM

⇒ Task #2 (0 / 0.67 pts.) - Related URLs

Progress: 100%

Details:

- Include the direct link to this file from the Milestone branch (should end in `.php` , , zero credit if it does not)
- Include direct link to the file on Render.com Prod (Just grab the base prod url and manually enter the path to the file)
- Include the related pull request link for this feature

URL #1

<https://rc728-it202-010-prod.onrender.com/project/login.php>



URL

<https://rc728-it202-010-prod.onrender.com/project/login.php>



URL #2

https://github.com/524RC/RC728-IT202-010/blob/prod/public_html/project/login.php



URL

https://github.com/524RC/RC728-IT202-010/blob/prod/public_html/project/login.php



URL #3

<https://github.com/524RC/RC728-IT202-010/pull/23>



URL

<https://github.com/524RC/RC728-IT202-010/pull/23>



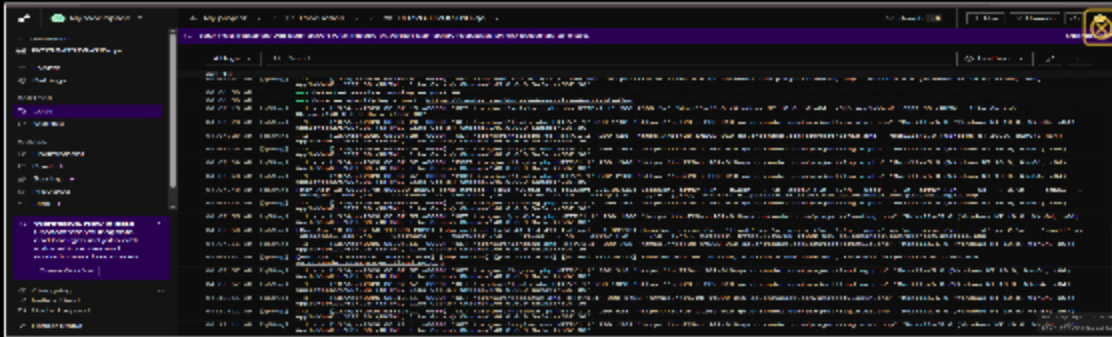
Saved: 4/13/2026 4:26:50 AM

Task #3 (0.6 / pts.) - User Session Evidence

Progress: 100%

Details:

- From the render.com logs (Logs tab), capture the session `error_log`
- It should show the id, username, email, and roles



this is the logs for render.com

 Saved: 4/13/2026 4:16:16 AM

Section #3: (1 pt.) Feature: User Will Be Able To Logout

Progress: 100%

Task #1 (1 pt.) - Evidence

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the successful logout message (visit the proper file on Render.com qa after a manual deploy)
- Show the code snippet of the logout page

```
public_html > project >  logout.php
1 <?php
2 // require functions.php to pull in flash()
3 require(__DIR__ . "/../lib/functions.php");
4 reset_session(); // clear session data and start a new session
5 flash("You have been logged out","success");
6 header("Location: " . get_url("login.php")); // redirect back to login
```

this shows the code for logout



this shows the logout message



Saved: 4/13/2026 4:29:55 AM

Part 2:

Progress: 100%

Details:

- Include the pull request url for this feature

URL #1

<https://github.com/524RC/RC728-IT202-010/pull/23>



URL

<https://github.com/524RC/RC728>



Saved: 4/13/2026 4:29:55 AM

Part 3:

Progress: 100%

Details:

- Briefly explain how the logout process works and how the user is officially logged off

Your Response:

The logout process works by user's session data using the `reset_session()` function. This removed all the stored session information and starts a new session which basically logs the user out since their authentication data is no longer stored. After doing this, a flash message is set to appear to give a logout message to the user to inform them that they have logged out and the user is redirected back to the login page. Without the session data, the user is no longer viewed as logged in which ends their session and basically logs them out.



Saved: 4/13/2026 4:29:55 AM

Section #4: (2 pts.) Feature: Security Rules

Progress: 100%

Task #1 (0 / 1 pt.) - Database Tables

Progress: 100%

Details:

- From the vs code mysql tool, show both the `Roles` and `UserRoles`

The screenshot shows the VS Code MySQL tool interface. The 'UserRoles' table is selected in the left sidebar. The main pane displays the table's data:

id	user id	role id	is active	created	modified
1	1	1	1	2026-04-13 08:44:11	2026-04-13 08:44:11
2	2	1	1	2026-04-13 08:44:11	2026-04-13 08:44:11

this shows userroles

The screenshot shows the VS Code MySQL tool interface. The 'Roles' table is selected in the left sidebar. The main pane displays the table's data:

id	name	description	is active	created	modified
1	Admin		1	2026-04-12 05:14:00	2026-04-12 05:14:00
1	not admin	you are not an admin	1	2026-04-12 05:44:10	2026-04-12 05:44:10

this shows roles

Saved: 4/13/2026 4:47:13 AM

Task #2 (1 pt.) - Demo Security Rules

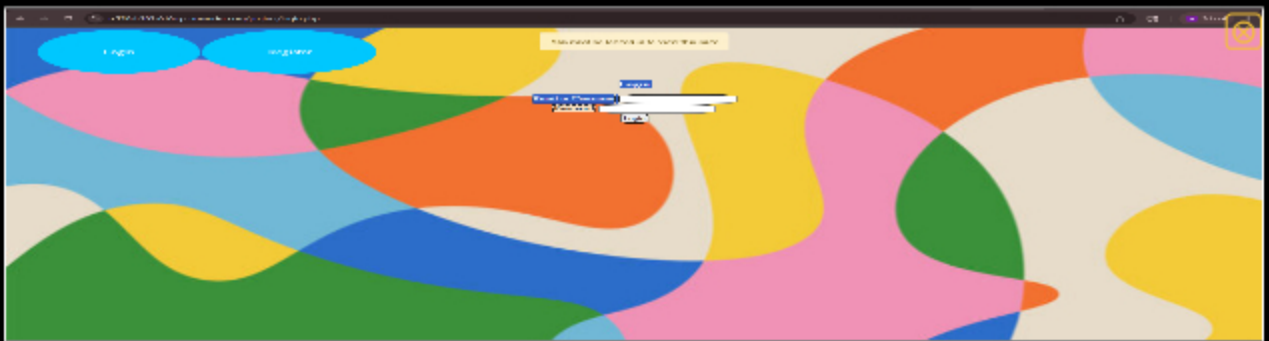
Progress: 100%

Part 1:

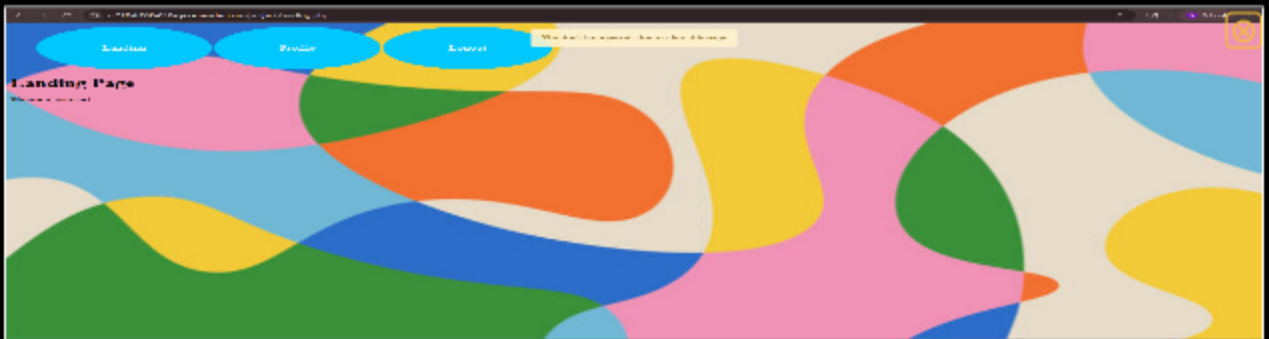
Progress: 100%

Details:

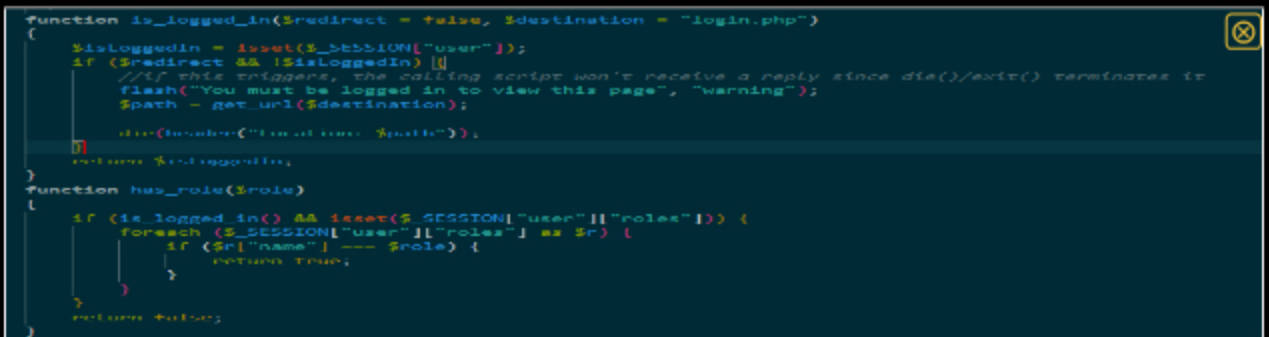
- Show the message related to needing to be logged in (i.e., try to manually
- Show the message related to not having the proper permission/role (i.e.,
- Include a snippet of the login check function
- Include a snippet of the role check function



this shows me trying to access the landing page while logged out



this shows me trying to access a page without the permission/role



this shows the code for the login check and the role check



Saved: 4/13/2026 7:34:27 PM

Part 2:

Progress: 100%

Details:

- Include the pull request url for this feature

URL #1

<https://github.com/524RC/RC728-IT202-010/pull/23>



URL

<https://github.com/524RC/RC728>



Saved: 4/13/2026 7:34:27 PM

⇒ Part 3:

Progress: 100%

Details:

- Briefly explain how the login check code works
- Briefly explain how the role check code works

Your Response:

the login check works by using the `is_logged_in()` function which checks if a user session exists by looking for the `$_SESSION["user"]` variable. if the user is not logged in and the `redirect` parameter is set to `true`, then the function will display a flash message giving the user a warning that they aren't logged in and can't view this page. This also redirects the user to the login page. This allows for unauthenticated users to not be able to access protected pages.

The role check works by using the `has_role()` function which first checks if the user is logged in, then it checks what the user's role is which is stored in the user's session. It loops through every role and compares them to the required role. If they match, then the function returns `true` allowing access to the page. If it doesn't find a match, then it returns `false` and prevents the user from accessing the restricted page.



Saved: 4/13/2026 7:34:27 PM

Section #5: (2 pts.) Feature: User Profile

Progress: 100%

≡ Task #1 (1 pt.) - Validation

Progress: 100%

Details:

- Show the relevant code snippets for each validation layer (html, js, php)
- Show the relevant demo of each validation layer (html, js, php)
- Briefly explain how the validation steps work at each layer

≡ Task #1 (0.33 pts.) - HTML Form/Validation

Progress: 100%

Details:

- System should allow the user to correct the error without wiping/clearing the form (for the PHP side this includes sticky-form logic to keep the username/email address entries)

Part 1:

Progress: 100%

Details:

- Show the code related to the profile page's form (HTML validation)
- Show examples of each validation message (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)

```
<div>Profile</div>
<form method="post" onsubmit="return validate(this);">
  <div class="mb-4">
    <label for="email">Email</label>
    <input type="text" name="email" id="email" required value="" />
  </div>
  <div class="mb-4">
    <label for="username">Username</label>
    <input type="text" name="username" id="username" required value="" />
  </div>
  <div class="mb-4">
    <label for="password">Current Password</label>
    <input type="password" name="currentpassword" id="cp" />
  </div>
  <div class="mb-4">
    <label for="password">New Password</label>
    <input type="password" name="newpassword" id="np" />
  </div>
  <div class="mb-4">
    <label for="password">Confirm Password</label>
    <input type="password" name="confirmpassword" id="conf" />
  </div>
  <input type="submit" value="Update Profile" name="save" />
</form>
```

this shows the html validation code

The screenshot shows the 'Profile' form with the following fields: Email, Username, Current Password, New Password, and Confirm Password. The Email field contains 'Rc728@' and has a red dashed border. A red error message box is displayed above the Username field, stating: 'Please enter a part following '@'. 'Rc728@' is incomplete.'

this shows an invalid email error

The screenshot shows the 'Profile' form with the following fields: Email, Username, Current Password, New Password, and Confirm Password. The Email field is empty and has a red dashed border. A red error message box is displayed above the Username field, stating: 'Please fill out this field.'

this shows an empty field error for email

The screenshot shows the 'Profile' form with the following fields: Email, Username, Current Password, New Password, and Confirm Password. The Email field contains 'Rc728@njit.edu' and has a red dashed border. A red error message box is displayed above the Username field, stating: 'Please fill out this field.'

this shows an empty field error for username

The screenshot shows a web form titled "Profile" with a close button in the top right. It contains five input fields: "Email" (with value "Rc728@njit.edu"), "Username" (with value "rc1"), "Password Reset" (a checkbox), "Current Password" (with masked characters "*****"), and "New Password" (with masked characters "****"). Below the "New Password" field, a validation message is displayed: "Please lengthen this text to 8 characters or more (you are currently using 4 characters)." The form is set against a colorful abstract background.

this shows an invalid password length error



Saved: 4/13/2026 8:54:04 PM

⇒ Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

The HTML validation for the profile works by using the required attribute that I added to ensure that the email and username fields aren't empty. The email field uses `type="email"` so the browser checks for a valid email format. The password fields use `minlength="8"` to make sure it has the minimum required length. These validations are handled by the browser and display these messages before submission to allow the user to fix the mistakes that they have. The form also uses sticky form logic which keep the values of the username and the email so that the user doesn't have to re-enter their information.



Saved: 4/13/2026 8:54:04 PM

≡ Task #2 (0.33 pts.) - JS Validation (validate() function)

Progress: 100%

🖼 Part 1:

Progress: 100%

Details:

- Show the code related to the profile page's `validate()` function
- Show examples of each validation message [username format, email format, password format, password doesn't match confirm] (you may be able to capture this in one or few screenshots) (visit the proper file on Dunder.com go after a manual

in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)

- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag

```

<script>
  $(document).ready(function() {
    // Email validation
    $('#email').on('blur', function() {
      if (!/^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/.test($('#email').val())) {
        $('#email').addClass('invalid');
      } else {
        $('#email').removeClass('invalid');
      }
    });

    // Username validation
    $('#username').on('blur', function() {
      if ($('#username').val().length < 3) {
        $('#username').addClass('invalid');
      } else {
        $('#username').removeClass('invalid');
      }
    });

    // Password validation
    $('#current_password').on('blur', function() {
      if ($('#current_password').val().length < 8) {
        $('#current_password').addClass('invalid');
      } else {
        $('#current_password').removeClass('invalid');
      }
    });

    $('#new_password').on('blur', function() {
      if ($('#new_password').val().length < 8) {
        $('#new_password').addClass('invalid');
      } else {
        $('#new_password').removeClass('invalid');
      }
    });

    $('#confirm_password').on('blur', function() {
      if ($('#new_password').val() !== $('#confirm_password').val()) {
        $('#confirm_password').addClass('invalid');
      } else {
        $('#confirm_password').removeClass('invalid');
      }
    });

    // Form submission
    $('#update_profile').on('click', function(e) {
      e.preventDefault();
      if ($('#email').hasClass('invalid') || $('#username').hasClass('invalid') || $('#current_password').hasClass('invalid') || $('#new_password').hasClass('invalid') || $('#confirm_password').hasClass('invalid')) {
        // Show error message
      } else {
        // Submit form
      }
    });
  });
</script>

```

This shows the js code with more validation steps added

Invalid email format.

List Roles

Profile

Email Rc72Ag2

Username rc1

Password Reset

Current Password

New Password

Confirm Password

Update Profile

this shows invalid email format warning from js

Username must be at least 3 characters

List Roles

Profile

Email Rc/286gnit.edu

Username rc

Password Reset

Current Password

New Password

Confirm Password

Update Profile

this show invalid username error

Password must be at least 8 characters long.

List Roles

Profile

Email Rc/286gnit.edu

Username rc1

Password Reset

Current Password

New Password

Confirm Password

Update Profile

this shows invalid password error

Password and Confirm password must match

List Roles

Profile

Email Rc/286gnit.edu

Username rc1

Password Reset

Current Password

New Password

Confirm Password

Update Profile

Profile

Email: Rc720q2njlt.edu

Username: rc1

Password Reset

Current Password:

New Password:

Confirm Password:

Update Profile

this shows password must match error



Saved: 4/13/2026 8:53:29 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

The JS validation is handled using the validation() function which is ran when the profile form is submitted. I chose to check and validate the email and username format, password length, and if the new password matches up with the confirm password. If any of these validations end up failing, a warning message is displayed on the page and the form submission is prevented. This allows for validating user input on the client side before the form is submitted allowing the user to correct any errors made.



Saved: 4/13/2026 8:53:29 PM

Task #3 (0.33 pts.) - PHP Validation (steps before the DB call)

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the code related to the profile page's PHP validation
- Show examples of each validation message [username format, email format, password format, invalid password, password doesn't match confirm, username taken, email taken] (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag and temporarily override the validate() function or use the provided helper script in the instructions


```

1 if (isset($_POST["email"], $_POST["username"])) {
2     $new_email = $_POST["email"];
3     $new_username = $_POST["username"];
4     $hasError = false;
5     // validate format
6     if (empty($new_email)) {
7         //echo "Email must not be empty<br>";
8         flash("Email must not be empty.", "danger");
9         $hasError = true;
10    }
11    // Sanitize and validate email
12    $new_email = sanitize_email($new_email);
13    if (!is_valid_email($new_email)) {
14        //echo "Invalid email address<br>";
15        flash("Invalid email address.", "danger");
16        $hasError = true;
17    }
18    if (!is_valid_username($new_username)) {
19        flash("Username must be lowercase, alphanumerical, and can only contain _ or -", "danger");
20        $hasError = true;
21    }
22 }

```

This shows the PHP validation for email and username

```

1 // Update user profile
2 if (isset($_POST["update_profile"])) {
3     // Sanitize and validate email
4     $new_email = sanitize_email($_POST["email"]);
5     if (!is_valid_email($new_email)) {
6         flash("Invalid email address.", "danger");
7         return;
8     }
9     // Sanitize and validate username
10    $new_username = sanitize_username($_POST["username"]);
11    if (!is_valid_username($new_username)) {
12        flash("Username must be lowercase, alphanumerical, and can only contain _ or -", "danger");
13        return;
14    }
15    // Update user profile
16    $user = User::find($_POST["id"]);
17    $user->email = $new_email;
18    $user->username = $new_username;
19    $user->save();
20    flash("Profile updated successfully.", "success");
21 }

```

This shows the code for updating the user profile and handling duplicate email or username errors

```

1 // Update password
2 if (isset($_POST["update_password"])) {
3     // Sanitize and validate password
4     $new_password = sanitize_password($_POST["password"]);
5     if (!is_valid_password($new_password)) {
6         flash("Password must be at least 8 characters long, contain at least one uppercase letter, one lowercase letter, one number, and one special character.", "danger");
7         return;
8     }
9     // Update password
10    $user = User::find($_POST["id"]);
11    $user->password = $new_password;
12    $user->save();
13    flash("Password updated successfully.", "success");
14 }

```

This shows the PHP validation for password updates

Invalid email address.

List Roles

Profile

Email: Rc728@njit.edu

Username: rc1

invalid email error

Username must be lowercase, alphanumerical, and can only contain _ or -

Role

List Roles

Assign Roles

Profile

Email: Rc728@njit.edu

Username: RC1

Email: Rc728@njit.edu
Username: rc1
Password Reset

invalid username error

Password must be at least 8 characters long
List Roles
Assign Roles

Profile

Email: Rc728@njit.edu
Username: rc1
Password Reset
Current Password:
New Password:
Confirm Password:
Update Profile

invalid password error

New passwords don't match
List Roles
Assign Roles

Profile

Email: Rc728@njit.edu
Username: rc1
Password Reset
Current Password:
New Password:
Confirm Password:
Update Profile

password mismatch error

Current password is invalid
List Roles
Assign Roles

Profile

Email: Rc728@njit.edu
Username: rc1
Password Reset
Current Password:
New Password:
Confirm Password:
Update Profile

invalid current password error

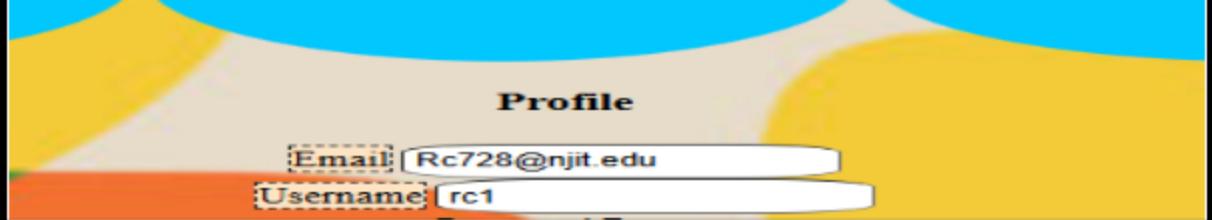
The chosen username is not available.
Password updated successfully
List Roles
Assign Roles

Profile

Email: Rc728@njit.edu
Username: rc1
Password Reset
Current Password:
New Password:
Confirm Password:
Update Profile

username taken error

The chosen email is not available.
List Roles
Assign Roles



email taken error



Saved: 4/13/2026 11:07:35 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

The PHP validation runs after the form is submitted and before any updates that can be made to the database. I validated the email and username format, password length is a minimum of 8 characters long, new and confirm passwords match, the current password is correct, and checks if theres a duplicate username and email in the database. If any of these validations fail, a flash message will be displayed and the update is cancelled. This allows the user to correct any errors made.



Saved: 4/13/2026 11:07:35 PM

Task #2 (0 / 1 pt.) - Related URLs

Progress: 100%

Details:

- Include the direct link to this file from the Milestone branch (should end in `.php` , zero credit if it does not)
- Include direct link to the file on Render.com Prod (Just grab the base prod url and manually enter the path to the file)
- Include the related pull request link for this feature

URL #1

<https://rc728-it202-010-prod.onrender.com/project/profile.php>



URL

<https://rc728-it202-010-prod.>



URL #2

https://github.com/524RC/RC728-IT202-010/blob/prod/public_html/



URL

https://github.com/524RC/RC728-IT202-010/blob/prod/public_html/



https://github.com/524RC/RC728-IT202-010/pull/23

URL #3

https://github.com/524RC/RC728-IT202-010/pull/23



URL

https://github.com/524RC/RC728-IT202-010/pull/23



Saved: 4/13/2026 11:27:26 PM

Section #6: (1 pt.) Misc

Progress: 100%

Task #1 (0.33 pts.) - Github Details

Progress: 100%

Part 1:

Progress: 100%

Details:

From the Commits tab of the Pull Request screenshot the commit history (it may not be possible to capture all, just grab a reasonable chunk)

Milestone1 fixed #23

Merged 524RC merged 4 commits into [main](#) from [Milestone1_fixed](#) yesterday

Conversation Commits Checks Files changed **28**

Commits on Apr 13, 2026

all the given files added and working	524RC committed 3 days ago	237c183		
everything fix and added all the given files	524RC committed 3 days ago	6882927		
fix validations and error without wiping	524RC committed yesterday	1374181		
Merge branch "fix" into Milestone1_fixed	524RC committed yesterday	9366118		

Commits tab

login validation finished #29

Merged 524RC merged 1 commit into [main](#) from [Milestone1_fixed](#) 19 hours ago

Conversation Commits **1** Checks Files changed **1**

Commits on Apr 13, 2026

login validation finished	524RC committed 19 hours ago	cb6a776		
---------------------------	------------------------------	---------	--	--

commits tab

every validation added #32

Merged 524RC merged 1 commit into [main](#) from [Milestone1_fixed](#) 7 minutes ago

Conversation Commits **1** Checks Files changed **4**

Commits on Apr 13, 2026

every validation added	524RC committed 7 minutes ago	020b4d2		
------------------------	-------------------------------	---------	--	--

commits tab

Milestone1 fixed #28 

 **Merged** 524RC merged 2 commits into **fix** from **Milestone1 fixed** 20 hours ago

 Conversation **0** |  Commits **2** |  Checks **0** |  Files changed **2** |  +101 -42 

 Commits on Apr 15, 2026

fix register validation
 524RC committed yesterday

 2005 / 29  <>

html and js validation added and working for login
 524RC committed yesterday

 1 file / 104  <>

commits tab

 Saved: 4/13/2026 11:31:12 PM

Part 2:

Progress: 100%

Details:

Include the link to the Pull Request (should end in `/pull/#`) (Milestone1 into qa)

URL #1

<https://github.com/524RC/RC728-IT202-010/pull/32/commits>



URL

<https://github.com/524RC/RC728>



URL #2

<https://github.com/524RC/RC728-IT202-010/pull/23>



URL

<https://github.com/524RC/RC728>



URL #3

<https://github.com/524RC/RC728-IT202-010/pull/29>



URL

<https://github.com/524RC/RC728>



URL #4

<https://github.com/524RC/RC728-IT202-010/pull/28>



URL

<https://github.com/524RC/RC728>



 Saved: 4/13/2026 11:31:12 PM

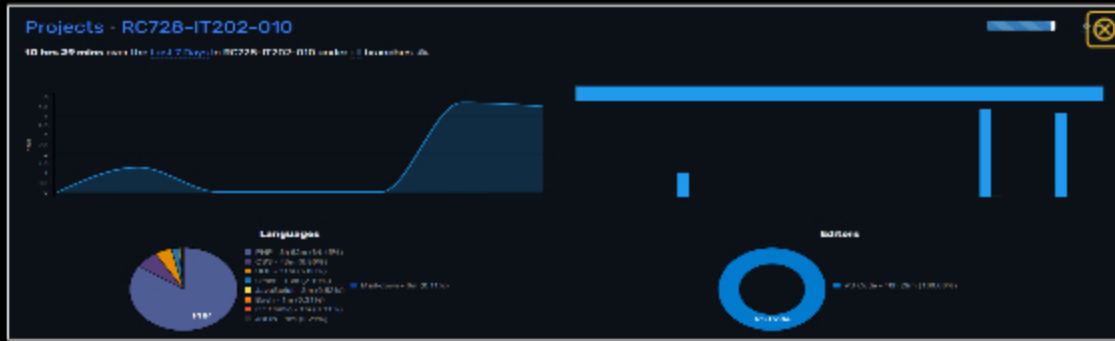
Task #2 (0 / 0.33 pts.) - WakaTime - Activity

Progress: 100%

Details:

- Visit the WakaTime.com Dashboard
- Click **Projects** and find your repository
- Capture the overall time at the top that includes the repository name

- Capture the individual time at the bottom that includes the repository name
- Note: The duration isn't relevant for the grade and the visual graphs aren't necessary



top part of waketime



bottom of waketime part 1



bottom of wakatiem part 2

Saved: 4/13/2026 11:33:54 PM

Task #3 (0.33 pts.) - Reflection

Progress: 100%

Task #1 (0 / 0.33 pts.) - What did you learn?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Briefly answer the question (at least a few decent sentences)

Your Response:

I learned alot about validations and why each validation is needed. I also learned about security and why we have certain validations and flash messages for these validations as to not give too much information away.



Saved: 4/13/2026 11:37:17 PM

⇒ Task #2 (0 / 0.33 pts.) - What was the easiest part of the assignment?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

The easiest part of the assignment was styling the website. This part was easy and fun to do and it didn't take too much thinking to do it. A lot of the css can be found online through w3schools. Other than styling the website the other easy part to the assignment was the sql part and assigning roles through the database.



Saved: 4/13/2026 11:39:25 PM

⇒ Task #3 (0 / 0.33 pts.) - What was the hardest part of the assignment?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

The hardest part of the assignment was getting each validation to work and running into a lot of errors. doing the first validation section was hard to figure out especially the JS and making the JS. Once i did the first one it was a lot easier to do the rest since they all ran through a similar format.



Saved: 4/13/2026 11:41:10 PM

